



Seminar

Engineering Scalable Systems: The Core Principles of LLD and SOLID

Presented by:

Prabhu Teja Pamula

24EMCA1237

Seminar Guide:

Dr. Brijendra Singh

Associate Professor

Zeroth Review

28 March 2026



Outline

1. Abstract and Problem Statement
2. Topic Selection and Relevance



Abstract and Problem Statement

Abstract

Modern software engineering requires a transition from basic functional code to modular, engineered systems. While High Level Design defines the broad system architecture, Low Level Design (LLD) ensures individual components remain maintainable and scalable. By implementing SOLID principles and industry standard Design Patterns, software can better withstand the complexities of production environments and evolving business requirements.

The Problem (Software Rot and Rigidity)

Production Fragility:

Minor updates in specific modules frequently trigger regression failures in unrelated services, making continuous deployment risky.

Architectural Rigidity:

Tightly coupled components create "ripple effects" where a single change forces a complete system refactor. This lack of isolation prevents individual classes from being tested or reused, making the logic too rigid to evolve without a total rewrite.

Scalability Bottlenecks:

Poor design choices at the class level lead to performance issues that cannot be solved by hardware alone, as the internal logic is not flexible enough to grow.

Accumulated Technical Debt:

Prioritizing "quick fixes" over design quality creates high long term maintenance costs, where the effort to fix bugs eventually outweighs new feature development.



Topic Selection and Relevance

Industry Standards

Major engineering firms across Big Tech, Fintech and Enterprise software prioritize Low Level Design throughout the software development lifecycle. Mastery of these principles is essential for writing clean, production ready code that minimizes technical debt and reduces long term maintenance costs in high stakes environments.

Scalability at the Core

System wide scalability is only as effective as its individual components. Effective design ensures that software logic is flexible enough to handle concurrent growth without requiring a total architectural rewrite.

Professional Development

This seminar bridges the gap between the academic MCA curriculum and real world software engineering practices, moving from basic functional coding to designing modular, maintainable systems.

Technical Depth

The study focuses on the practical application of SOLID principles and Design Patterns to eliminate common architectural bottlenecks, ensuring the system remains "open for extension but closed for modification."



Thank You

Questions and Feedback